Network Backend Roadmap

Protocols to build from scratch, projects to ship, and the skills job postings ask for

Target role: backend developer in networking · Languages: Go, Rust, C++ · Prepared 6 November 2026

20	3 + 2	4	3
protocols to implement from scratch	core projects + optional projects	levels, from TFTP to RADIUS and SIP	languages: Go, Rust, C++

1. How to use this plan

- **The rule.** Implement each protocol from its RFC, then verify it with real tools (*dig, curl, tcpdump, Wireshark, FRR, SIPp, radtest*). That lets you say “my server answers dig” instead of “I know DNS”.
- **Language order.** Start with **Go**: fast results, and it is named in most postings. Add **Rust** after 3–4 weeks, on a protocol you already know (for example DNS ported from Go). Treat **C++** as a third language if you target Motorola, Ericsson or Cisco roles that ask for it.
- **Quality over quantity.** Every protocol gets tests on real packets, parser fuzzing, a benchmark and a short README. Do not write cryptography from scratch: take TLS, QUIC and SSH from libraries.

Legend. In the language column the first language is recommended and the others are reasonable alternatives. Python = prototypes, tests and scripts. Node/TS = Node.js and TypeScript.

2. Protocols to implement from scratch

Protocol (RFC)	Languages	What to build	Verify with	Useful for
LEVEL 1 · Foundations				
1. TFTP **RFC 1350**	**Go** · Rust · C++ · Python	UDP client and server: RRQ/WRQ, 512-byte blocks, ACKs, retransmission and timeouts. A good warm-up.	tftp-hpa, Wireshark	UDP, state machines, timeouts
2. TCP and HTTP/1.1 **RFC 9293, 9112**	**Go** (no net/http) · Rust · C++	TCP server with framing and timeouts, then your own HTTP server: request parser, headers, keep-alive, chunked encoding, a few routes.	curl, wrk, k6, Wireshark	Proxies, APIs, any backend
3. WebSocket **RFC 6455**	**Go** · Rust · Node/TS	Upgrade handshake, frames, masking, ping/pong, close handshake.	websocat, browser	Dashboards, UniFi, notifications
4. DNS **RFC 1034, 1035, 6891 (EDNS)**	**Go**, then port to **Rust** · C++	Message parser and builder with name compression, stub resolver, authoritative server over UDP and TCP, TTL cache.	dig, kdig, Wireshark	Cloudflare, ISPs, project 3
5. DHCP **RFC 2131, 2132**	**Go** · Rust · C++	Server with an address pool, leases and options, and a REST API for management (IPAM).	dhclient in a network namespace, Wireshark	ISPs, networking, IPAM
LEVEL 2 · Packets and lower layers				
6. Ethernet, ARP, IPv4, ICMP, UDP, TCP **RFC 826, 791, 792, 768, 9293**	**Rust** · Go (gopacket) · C++ (libpcap)	Raw-byte parser that reads pcap files, your own ping and traceroute, ARP spoofing detection.	Wireshark, tcpdump	Packet processing, eBPF, IDS, security
7. NTP **RFC 5905**	**Go** · Rust · C++	SNTP client and a simple server: offset, delay, modes, stratum.	chronyc, ntpdate -q	Systems, time telemetry

Protocol (RFC)	Languages	What to build	Verify with	Useful for
8. STUN **RFC 8489**	**Go** · Rust · Node/TS	Client and server: XOR-MAPPED-ADDRESS, NAT type detection.	Wireshark, public STUN server	WebRTC, VoIP
9. TLS 1.3 (ClientHello only) **RFC 8446**	**Rust** · Go	Handshake parser: SNI, ALPN, versions, JA3 fingerprint. No cryptography to write.	openssl s_client, curl -v	Security, traffic filtering
10. Reliable transport over UDP **(inspired by RFC 9293, 9000)**	**Rust** · Go · C++	Sliding window, retransmission, congestion control, simulated packet loss.	tc netem, iperf3	How TCP and QUIC work inside, layer 2
11. GRE and VXLAN **RFC 2784, 7348**	**Rust** · Go · C++	Encapsulation over a TUN device, two hosts in separate network namespaces.	ip, tcpdump	SD-WAN, VPN, cloud
LEVEL 3 · Routing and telemetry				
12. BGP-4 **RFC 4271 (BMP: RFC 7854)**	**Go** · Rust · C++	State machine, OPEN, KEEPALIVE, UPDATE, NOTIFICATION, route table. Peer with FRR in containerlab.	FRR (vtysh), Wireshark	Cloudflare, carriers, project 2
13. OSPFv2 (advanced) **RFC 2328**	**Go** · Rust	Hello, adjacencies, type-1 LSAs, SPF algorithm.	FRR, Wireshark	Routing, carrier networks
14. SNMP v2c **RFC 3416**	**Go** · Python · Rust	Agent with a few OIDs and a manager: GET, GETNEXT, WALK, BER encoding.	snmpwalk, snmpget	Monitoring, ISPs
15. sFlow v5 / IPFIX **RFC 7011**	**Go** · Rust	Datagram collector, flow aggregation, storage in ClickHouse.	hsflowd, softflowd	Telemetry, project 2
16. Syslog **RFC 5424**	**Go** · Rust	UDP and TCP receiver, parser, index, simple queries.	logger, rsyslog	Observability, Splunk
LEVEL 4 · ISP and telecom				
17. RADIUS **RFC 2865, 2866 (accounting)**	**Go** · Rust · C++ · Java	Server for Access-Request / Accept / Reject, attributes, Message-Authenticator, accounting. Subscribers in PostgreSQL, traffic accounting into ClickHouse.	radtest, eapol_test, FreeRADIUS as client	ISPs, Wi-Fi, UniFi
18. SIP + SDP **RFC 3261, 8866**	**Go** · Rust · Node/TS · C++ (PJSIP)	Parser, REGISTER with digest authentication, INVITE / 200 / ACK / BYE, user location (Redis), call routing.	SIPp, Linphone, Wireshark	VoIP, Ubiquiti Talk, Cisco, Motorola
19. RTP / RTCP **RFC 3550**	**Go** · Rust · C++	Receiver, jitter buffer, loss and jitter statistics, a simple RTP relay.	SIPp (media), ffmpeg, Wireshark	Media, telecom, WebRTC
20. MQTT 3.1.1 / 5.0 **OASIS specification**	**Go** · Rust · Node/TS	Broker: CONNECT, PUBLISH, SUBSCRIBE, QoS 0 and 1, retained messages.	mosquitto_pub / mosquitto_sub	IoT, UniFi Connect

LEVEL 5 · Use libraries (do not write from scratch)

What	Libraries	Why not from scratch
QUIC and HTTP/3	quic-go (Go), quinn (Rust)	A huge specification; what you learn in item 10 is enough
TLS	crypto/tls (Go), rustls (Rust)	Never write your own cryptography; it is a security risk
SSH and NETCONF	x/crypto/ssh (Go), russh (Rust)	Cryptography. Focus on the client and configuration automation
gRPC and gNMI	grpc-go, tonic (Rust)	It is enough to design the API in protobuf
WireGuard / Noise	wireguard-go, boringtun	Build the tunnel from item 11, but take the encryption from a library

3. Core projects (one repository each)

The three projects turn the protocols above into the systems that job postings ask about. Each has milestones (MVP, v1, v2), so you can show a project after the first one.

Project 1 · edge-proxy: reverse proxy with a control plane	Go · Rust · C++
Goal	Build an L7 proxy that you steer through an API, the way API gateways and CDNs do.
Components	**Data plane:** HTTP/1.1 and HTTP/2 proxy with TLS termination, load balancing (round-robin, least-connections), health checks, rate limiting (token bucket), circuit breaker, timeouts. **Control plane:** REST and gRPC for managing routes, upstreams, certificates and policies. **Data:** configuration in PostgreSQL, rate-limit counters in Redis.
Technology	Prometheus and Grafana, OAuth2 / JWT on the API, Docker, Kubernetes (kind), Helm, GitHub Actions, k6 load tests.
Languages	**Go** (primary, whole system). **Rust** (tokio, hyper, axum): an alternative, or the data plane in v2. **C++** (Boost.Beast, Asio): possible, but more work.
Milestones	**MVP:** HTTP/1.1 proxy with round-robin and health checks, config from a file. **v1:** REST control plane, PostgreSQL, config reload without dropping connections, Redis rate limiting, metrics. **v2:** TLS, HTTP/2, JWT, Helm and Kubernetes, k6 test, outage runbook.
Done when	Throughput and p99 latency are measured (numbers in the README). Config reload does not interrupt requests. When an upstream goes down, traffic shifts to healthy instances. Tests and parser fuzzing are in place.
Covers	Cloudflare (TLS/HTTP proxy, APIs, Go), Akamai (API gateway, rate limiting, load balancing, Kubernetes), Ubiquiti (OAuth, Docker, SQL, defensive programming).

Project 2 · netops-platform: network telemetry and configuration	Go · Rust · Python
Goal	Collect data from network devices, store and version their configuration, and alert on failures.
Components	**Lab:** containerlab with 3–4 FRR routers (BGP, OSPF). **Collectors:** SNMP, sFlow, configuration over SSH and NETCONF, BGP monitor (the session from item 12, or BMP). **Processing:** NATS or Kafka queue, aggregator. **Data:** ClickHouse (flows), PostgreSQL (inventory, configuration history with diff and rollback). REST and gRPC API, alert rules.
Technology	Grafana, Prometheus, Docker Compose or Kubernetes, GitHub Actions, integration tests on the lab.
Languages	**Go** (primary). **Rust** (sFlow and SNMP collectors). **Python** (lab scripts, integration tests). **C++** is possible for collectors.
Milestones	**MVP:** device inventory, config fetch over SSH, versioning and diff. **v1:** SNMP, metrics, Grafana dashboard. **v2:** sFlow → NATS → ClickHouse, BGP monitor, alerts.
Done when	Taking a lab link down raises an alert in under 30 s. Configuration history with diff and rollback. A flow chart built from ClickHouse. One command brings up the whole lab.
Covers	Cloudflare (Traffic and Network Platforms teams: ClickHouse, NETCONF, Grafana, PostgreSQL, BGP), Cisco, Arista, Juniper (network automation and telemetry).

Project 3 · dns-xdp: DNS server with in-kernel protection	Rust · Go · C
Goal	Build a DNS server and a Linux kernel packet filter that limits traffic before it reaches the application.
Components	**Server:** authoritative from zone files plus a cache, UDP and TCP, EDNS, response size limit. **Filter:** XDP/eBPF program that rate-limits packets per source address and counts statistics in BPF maps. **Measurements:** traffic generator (dnsperf), packets-per-second chart, metrics in Grafana.
Technology	eBPF/XDP, BPF maps, tokio, Prometheus, Docker, fuzzing (cargo-fuzz), CI.
Languages	**Rust** (primary: tokio plus aya). **Go** (cilium/ebpf). **C** / **C++** with libbpf: closest to the kernel, if you target C/C++ companies.
Milestones	**MVP:** server from zone files, parser ported from Go. **v1:** cache, TCP, EDNS, metrics. **v2:** XDP filter, throughput tests with and without it.
Done when	A packets-per-second chart with and without XDP. The DNS parser survives fuzzing. The server answers dig for test zones. A short write-up of what happens under a query flood.
Covers	Cloudflare (DNS, eBPF/XDP, Rust, systems programming), networking companies that use C/C++ (Motorola, Ericsson).

4. Optional projects (for a specific target)

Option A · isp-provisioning: subscriber backend for an ISP	Go · Rust · C++
Goal	Simulate what an ISP backend does: subscriber login, address assignment and usage accounting.
Components	RADIUS server (RFC 2865, 2866) with subscribers in PostgreSQL and speed plans passed as RADIUS attributes. The DHCP server from item 5. Session accounting into ClickHouse, usage reports, REST for a portal. Optionally Change of Authorization (RFC 5176).
Technology	PostgreSQL, ClickHouse, Docker, Grafana, FreeRADIUS as a test client.
Languages	**Go** (primary). **Rust**, **C++**. **Java** is common in operator OSS/BSS systems, but that is general knowledge, not taken from postings I checked.
Milestones	**MVP:** Access-Request checked against the subscriber database. **v1:** accounting and reports. **v2:** plans and CoA.
Done when	radtest and eapol_test pass. A usage report for a subscriber. A load test: authentications per second.
Covers	ISP and Wi-Fi companies, Ubiquiti (UniFi).

Option B · voip-gateway: SIP registrar with an RTP relay	Go · Node/TS · Rust
Goal	Build a small VoIP system: registration, call setup and media relay.
Components	SIP registrar and proxy (item 18) with digest authentication, user location in Redis and routing rules. RTP relay (item 19) with jitter and loss statistics. Optional WebRTC gateway. REST for management and call detail record (CDR) export.
Technology	Redis, PostgreSQL, SIPp for tests, Wireshark, Docker.
Languages	**Go** (Pion library for WebRTC). **Node/TS** (Ubiquiti UniFi Talk uses Node.js). **Rust**. **C++** (PJSIP).
Milestones	**MVP:** REGISTER and INVITE between two clients. **v1:** RTP relay and statistics. **v2:** WebRTC and CDR.
Done when	A call between two Linphone clients through your server. A SIPp scenario running in CI. A jitter and loss chart.
Covers	Ubiquiti (UniFi Talk), Cisco (Webex), Motorola (telecom).

5. Which job requirement you practise where

Requirement in postings	Where you practise it	Companies
Go and/or Rust	All projects. Go: 1 and 2. Rust: 3	Cloudflare, Ubiquiti
TCP/UDP, DNS, TLS, HTTP, QUIC	Protocols 2, 4, 9, 10; projects 1 and 3	Cloudflare
eBPF / XDP, packet processing	Protocol 6; project 3	Cloudflare
BGP, routing, NETCONF, sFlow	Protocols 12, 13, 15; project 2	Cloudflare, Cisco, Arista
REST and gRPC APIs, distributed systems	Projects 1 and 2	All
PostgreSQL, ClickHouse, Redis	Projects 1, 2 and option A	Cloudflare, Ubiquiti
Docker, Kubernetes, CI/CD	Projects 1 and 2	Akamai, Cloudflare, Ubiquiti
Prometheus, Grafana, observability	All projects	Cloudflare, Akamai, Cisco
OAuth, OWASP, API security	Project 1 (JWT, OAuth2)	Ubiquiti, Cisco
SIP, RTP, WebRTC	Protocols 18 and 19; option B	Ubiquiti, Cisco, Motorola
On-call, owning what you build	Runbook and alerts in projects 1 and 2	Cloudflare, Cisco

6. Checklist for every repository

- **README:** what it is and why, an architecture diagram, one-command start, benchmark results with numbers, and a “limitations” section.
- **CI on GitHub Actions:** tests, lint, build and a green badge.
- **Tests:** unit tests, tests on real captured packets, and integration tests. Fuzz every parser (*go test -fuzz*, *cargo-fuzz*).
- **Repository layout:** three separate repositories (edge-proxy, netops-platform, dns-xdp) plus one shared *protocols-from-scratch* with go/ and rust/ folders, where each protocol has its own README.
- **Commit history:** small, descriptive commits. A license (for example MIT) and GitHub topics.
- **Honesty:** say you know a technology only when the project works and is on GitHub. Interviewers ask for details: why this way, how you measured, what went wrong. If you use AI coding tools, understand every line.

Note: the requirements in this plan come from job postings, most of which are now closed (Cloudflare, Ubiquiti, Akamai). Treat them as a pattern of expectations, not as current openings. The ISP part is based on general knowledge, not on postings I analysed.